HARAMAYA UNIVERSITY | DEPARTMENT OF COMPUTER SCIENCE

CoSc3053: 90-DAY JAVA MASTERY CHALLENGE

DAY 2 WORKSHEET: CONSOLE GRADE CALCULATOR (GradeCalc)

Student Name: _____	**Date: _____**
Student ID: _____	**IDE: IntelliJ IDEA 2024+**
Section / Group: _____	**Evaluation Score: ____ / 10.0**

1. Day 2 Challenge Objective & Core Concepts

The primary goal of the Day 2 Challenge is to master Java syntax, primitive data types, input validation using repetition loops, multi-way conditional decision ladders, and modern arrow switch expressions. You will construct a robust, production-grade console application named **`GradeCalc.java`** that evaluates student marks with complete input safety.

Input Validation Loop	Uses a do-while loop to continuously re-prompt the user until a score in the valid range [0.0 - 100.0] is entered.
Multi-Way Selection	Executes an if-else-if ladder to categorize the numeric mark into standard letter grades (A, B, C, D, F).
Modern Switch Expression	Employs JDK 14+ arrow syntax (->) to yield academic remarks without fall-through bugs.
Robust Input Guarding	Utilizes Scanner.hasNextDouble() to prevent crashes when non-numeric tokens are entered.

2. Executable Java Source Code: GradeCalc.java

Type or copy the following validated Java program into your IDE under the package **`edu.haramaya.cosc3053.day2`**:

```
package edu.haramaya.cosc3053.day2;

import java.util.Scanner;

/**
 * Day 2 Challenge: Console Grade Calculator Engine
 * Demonstrates input validation, if-else decision ladders, and modern switch expressions.
 */
public class GradeCalc {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double mark = -1.0;
```

```
        System.out.println("========================================");
        System.out.println("   HARAMAYA CS: GRADE CALCULATOR ENGINE  ");
        System.out.println("========================================");

        // 1. Input Validation Loop (Reprompts if mark < 0 or mark > 100)
        do {
            System.out.print("Enter exam mark [0.0 - 100.0]: ");
            if (scanner.hasNextDouble()) {
                mark = scanner.nextDouble();
                if (mark < 0.0 || mark > 100.0) {
                    System.out.println("❌ Invalid Score! Mark must be between 0.0 and 100.0.\n");
                }
            } else {
                System.out.println("❌ Invalid Input! Please enter a valid numeric value.\n");
                scanner.next(); // Clear non-numeric token
            }
        } while (mark < 0.0 || mark > 100.0);

        // 2. Multi-Way Selection Ladder (if-else-if)
        char letterGrade;
        if (mark >= 90.0) {
            letterGrade = 'A';
        } else if (mark >= 80.0) {
            letterGrade = 'B';
        } else if (mark >= 70.0) {
            letterGrade = 'C';
        } else if (mark >= 60.0) {
            letterGrade = 'D';
        } else {
            letterGrade = 'F';
        }

        // 3. Modern Switch Expression with Arrow Syntax (JDK 14+)
        String remarks = switch (letterGrade) {
            case 'A' -> "Distinction! Outstanding academic performance.";
            case 'B' -> "Very Good! Upper division pass.";
            case 'C' -> "Satisfactory Pass.";
            case 'D' -> "Conditional Pass. Support recommended.";
            case 'F' -> "Unsatisfactory. Academic re-evaluation required.";
            default  -> "Unknown Grade State.";
        };

        // 4. Pass / Fail Decision Logic
        boolean isPassed = (letterGrade != 'F');

        // 5. Output Results Summary
        System.out.println("\n----------------------------------------");
        System.out.println("           FINAL EVALUATION RESULT        ");
        System.out.println("----------------------------------------");
        System.out.printf(" Exam Mark Score : %.2f / 100.00%n", mark);
        System.out.println(" Letter Grade    : " + letterGrade);
        System.out.println(" Academic Status : " + (isPassed ? "PASSED ☑" : "FAILED ❌"));
        System.out.println(" Instructor Remark: " + remarks);
        System.out.println("========================================");

        scanner.close();
    }
}
```

3. Visual Execution Snapshots & Console Logs

Visual Snapshot 3.1: Successful Execution (Valid Mark Input)

```
📥 SIMULATED KEYBOARD INPUT:
92.5

========================================
   HARAMAYA CS: GRADE CALCULATOR ENGINE
========================================
Enter exam mark [0.0 - 100.0]: 92.5

----------------------------------------
           FINAL EVALUATION RESULT
----------------------------------------
 Exam Mark Score : 92.50 / 100.00
 Letter Grade    : A
 Academic Status : PASSED ☑
 Instructor Remark: Distinction! Outstanding academic performance.
========================================
```

Visual Snapshot 3.2: Re-prompt Recovery (Invalid Out-of-Bounds Score)

```
📥 SIMULATED KEYBOARD INPUT SEQUENCE:
105.0  (Out of bounds -> Trigger error)
-15.0   (Out of bounds -> Trigger error)
74.0    (Valid C grade score)

========================================
   HARAMAYA CS: GRADE CALCULATOR ENGINE
========================================
Enter exam mark [0.0 - 100.0]: 105.0
❌ Invalid Score! Mark must be between 0.0 and 100.0.

Enter exam mark [0.0 - 100.0]: -15.0
❌ Invalid Score! Mark must be between 0.0 and 100.0.

Enter exam mark [0.0 - 100.0]: 74.0

----------------------------------------
           FINAL EVALUATION RESULT
----------------------------------------
 Exam Mark Score : 74.00 / 100.00
 Letter Grade    : C
 Academic Status : PASSED ☑
 Instructor Remark: Satisfactory Pass.
========================================
```

4. Self-Guided Checkpoint & Reflection Tasks

Task 4.1: Repetition Mechanics

Why is a do-while loop superior to a standard while loop for user input validation in command-line applications? Explain how post-test evaluation guarantees at least one execution prompt.

Student Analysis / Response:

Task 4.2: Modern Arrow Switch vs. Legacy Switch

Contrast the arrow syntax switch expression (switch (letterGrade) { case 'A' -> ... }) with traditional case blocks. Highlight how arrow syntax eliminates the need for break statements and prevents accidental fall-through bugs.

Student Analysis / Response:

Task 4.3: Input Safety Extension

Modify GradeCalc.java so that if a student inputs non-numeric characters (e.g., 'eighty'), Scanner.hasNextDouble() catches the error cleanly without throwing an InputMismatchException crash.

Student Analysis / Response:

Task 4.4: Boundary Testing

Test the boundary mark 89.99 versus 90.00 in your IDE. State which letter grade each receives and explain how floating-point precision operators handle conditional boundary comparisons (>= vs >).

Student Analysis / Response:

5. Evaluation Rubric & Sign-Off

Evaluation Criteria	Performance Benchmark	Score
Input Validation Robustness	do-while loop correctly handles out-of-bounds numbers and non-numeric inputs without crashing	**/ 2.5**
Selection Decision Logic	if-else-if decision ladder correctly maps numeric boundaries to letter grades (A-F)	**/ 2.5**
Modern Switch Syntax	Arrow switch expression (->) correctly formats academic remarks without fall-through errors	**/ 2.0**
Checkpoint Reflection Quality	Thorough student responses explaining loop choice, switch comparison, and boundary tests	**/ 2.0**
Code Formatting & Conventions	Proper camelCase naming, clear console formatting, package organization, and comments	**/ 1.0**